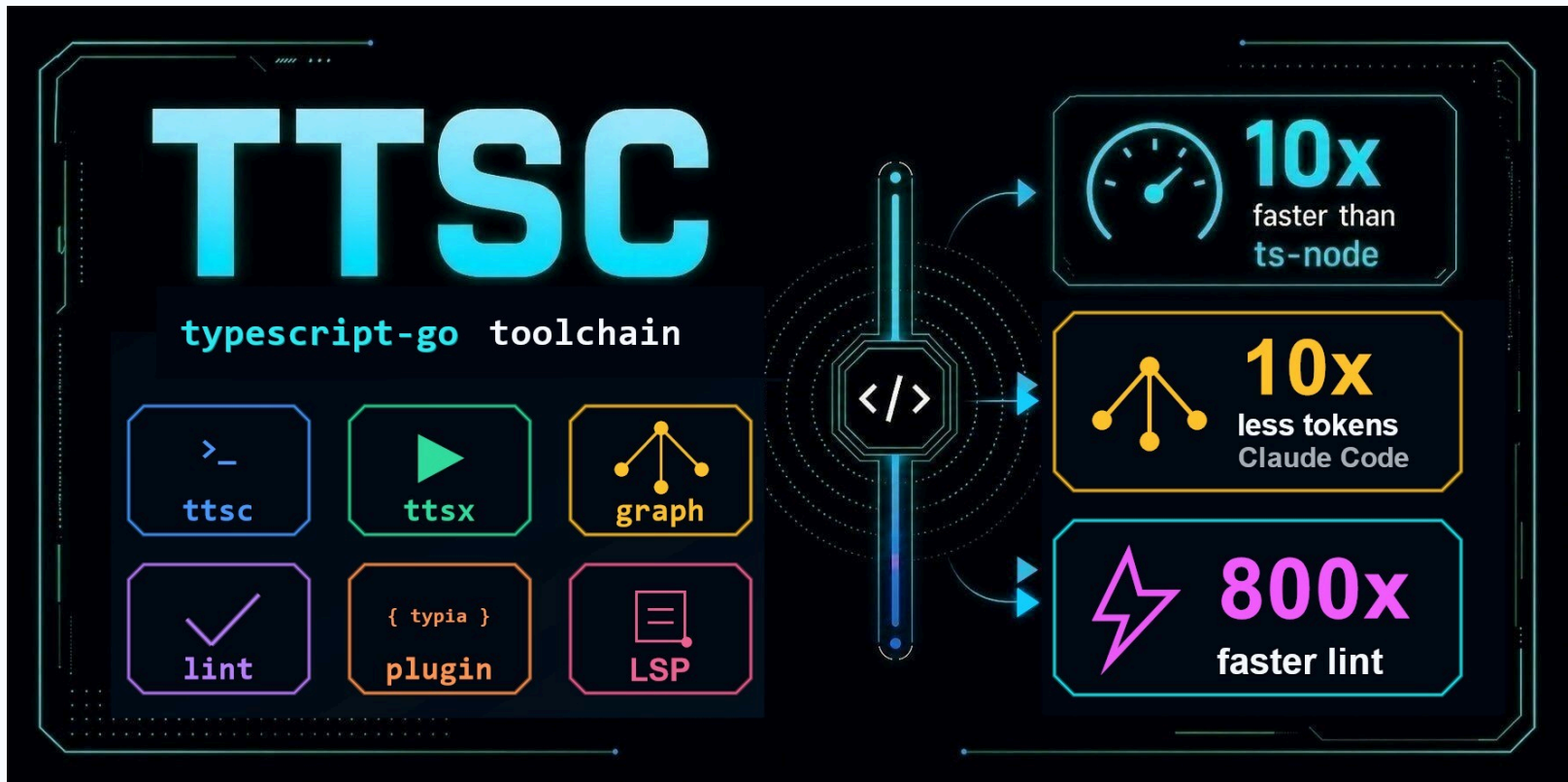


TTSC

From Transformer Crisis to TypeScript-Go Toolchain

TypeScript Backend Meetup

Samchon, 2026-06-26



- <https://ttsc.dev>
- <https://github.com/samchon/ttsc>

TL;DR

- Typia and Nestia
 - pure TypeScript input
 - generated runtime/tooling output
- TypeScript-Go
 - faster compiler
 - missing JavaScript patch point
- TTSC
 - transformer host
 - compiler-state toolchain

Index

1. Typia and Nestia
2. TypeScript-Go
3. Transformer Survival
4. Toolchain Opportunity

1. Typia and Nestia

1.1. Typia

1.2. Nestia

1.3. Common Engine

1.4. Transformer

1.1. Typia

- Generic call
 - type argument
 - erased later
- Compiler output
 - direct predicate
 - plain JavaScript

```
//——  
// TypeScript  
//——  
const isString = typia.createIs<string>();  
  
//——  
// JavaScript (Compiled)  
//——  
const isString = (input) => "string" === typeof input;
```

1.1. Typia

- User writes
 - TS type
 - tags
 - generic call
- No duplicate
 - no schema
 - no decorator
 - no DTO class

```
import typia, { tags } from "typia";

interface IMember {
  id: string & tags.Format<"uuid">;
  email: string & tags.Format<"email">;
  age: number &
    tags.Type<"uint32"> &
    tags.ExclusiveMinimum<19> &
    tags.Maximum<100>;
}

const isMember =
  typia.createIs<IMember>();
```

1.1. Typia

- Compiler emits
 - UUID check
 - email check
 - uint32 check
 - range check
- Erased type
 - JS
 - no interpreter

```
const isMember = (input) =>
  "object" === typeof input &&
  null !== input &&
  "string" === typeof input.id &&
  _isFormatUuid(input.id) &&
  "string" === typeof input.email &&
  _isFormatEmail(input.email) &&
  "number" === typeof input.age &&
  _isTypeUint32(input.age) &&
  19 < input.age &&
  input.age ≤ 100;
```

1.1. Typia

- Runtime validation
 - 20,000x
 - class-validator
 - detailed failure paths
- JSON serialization
 - 200x
 - class-transformer
 - type-specialized stringifier
- Same source
 - TypeScript type
 - JSDoc
 - tags

Typia speedup

Log-scaled by multiplier. Baseline target = 1x.

Runtime validation



JSON serialization



1.2. Nestia

- Backend code
 - path
 - method
 - parameter
 - body type
 - return
- Contract source
 - controller

```
@Controller("bbs/:section/articles")
export class BbsArticlesController {
  @TypedRoute.Post()
  public async create(
    @TypedParam("section") section: string,
    @TypedBody() input: IBbsArticle.ICreate,
  ): Promise<IBbsArticle> {
    return this.service.create(section, input);
  }
}
```

1.2. Nestia

- Generated SDK
 - typed fetch
 - DTO
 - checks
- Backend
 - no interface copy
 - FE API

```
const article: IBbsArticle =  
  await api.functional.bbs.articles.create(  
    connection,  
    "general",  
    {  
      title: "Hello World",  
      body: "My first article",  
    } satisfies IBbsArticle.ICreate,  
  );
```

1.2. Nestia

- Server performance
 - 10x+ total path
 - 30x Fastify path
 - validation 20,000x
 - serialization 200x
- SDK generation
 - typed fetch functions
 - DTO structures
 - npm distribution
- Tooling
 - OpenAPI
 - mockup simulator
 - E2E test functions
 - Swagger editor

Nestia speedup

Same log scale as typia. Baseline target = 1x.

Server path



Validation



Serialization



1.3. Common Engine

- typia
 - TypeScript type
 - Runtime validator
 - JSON serializer
 - LLM schema
- nestia
 - NestJS controller
 - Runtime boundary
 - client SDK
 - OpenAPI document
- Same shape: compiler analysis, generated artifacts

1.3. Common Engine

- Input
 - TypeScript source
 - types
 - decorators
 - JSDoc
- Compiler facts
 - AST
 - Checker
 - symbols
 - diagnostics

1.4. Transformer

- Transformer
 - compile-time code generation
 - from TypeScript compiler facts
- Reads
 - source file
 - AST
 - Checker
- Re-writes
 - JavaScript emit
 - diagnostics
 - generated artifacts

1.4. Transformer

- Project setup
 - install `ts-patch`
 - hack TypeScript
- Compiler config
 - transformer path

```
// tsconfig.json
{
  "compilerOptions": {
    "plugins": [
      { "transform": "typia/lib/transform" },
      { "transform": "@nestia/core/lib/transform" },
      { "transform": "@nestia/sdk/lib/transform" }
    ]
  }
}

// package.json
{
  "scripts": {
    "prepare": "ts-patch install"
  },
  "devDependencies": {
    "ts-patch": "^3.2.1"
  }
}
```

1.4. Transformer

- Public API
 - still TypeScript
 - still npm package
 - still framework-friendly
- Private engine
 - JavaScript compiler process
 - transformer host
 - emit pipeline
- Key split
 - language compatibility
 - plugin compatibility

2. TypeScript-Go

- 2.1. Native Compiler
- 2.2. Patch Model
- 2.3. Broken Assumption
- 2.4. Project Risk

2.1. Native Compiler

User-visible promise

- same TypeScript source
- same project files
- faster check loop

Implementation change

- compiler rewritten in Go
- language service moves native
- JavaScript package is no longer the center

First reaction

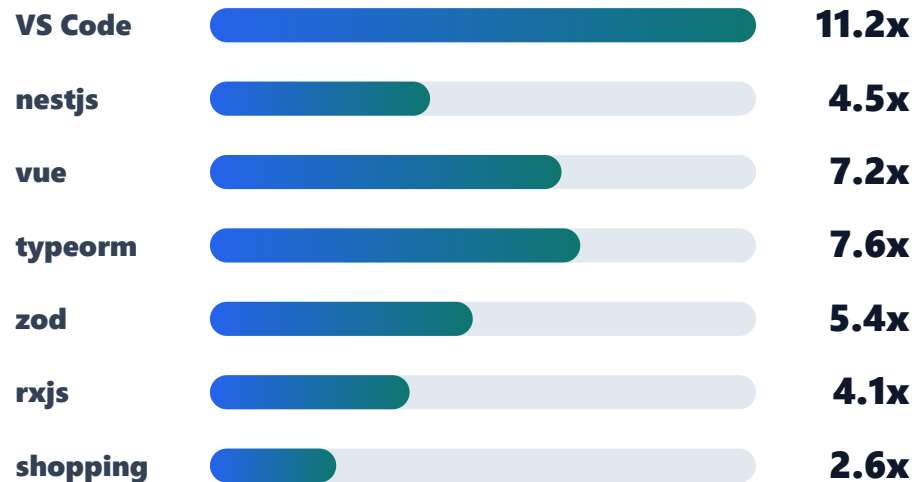
- this is excellent news

2.1. Native Compiler

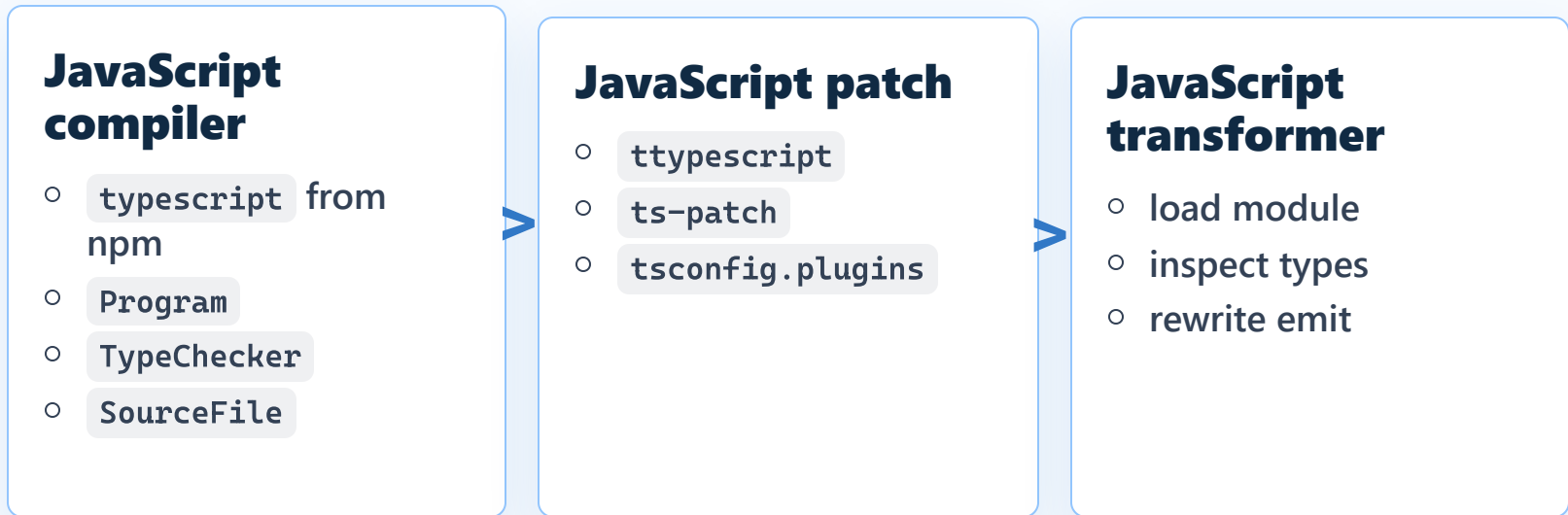
- Benchmark headline
 - VS Code: **11.2x**
 - 73.29s to 6.54s
- Pattern
 - fixture set repeats the win
 - speed comes from the native compiler base

TypeScript-Go type-check speedup

Multiplier only. VS Code checkers: 73.29s to 6.54s.



2.2. Patch Model



2.2. Patch Model

What users configured

- `ts-patch install`
- `plugins` in `tsconfig.json`
- transformer package path

What tools assumed

- patched TypeScript is executable
- JavaScript module can enter emit
- compiler API objects are reachable

What typia and nestia used

- type facts before erasure
- controller facts before runtime
- generated code during emit

2.3. Broken Assumption

```
{
  "compilerOptions": {
    "plugins": [
      { "transform": "typia/lib/transform" },
      { "transform": "@nestia/core/lib/transform" },
      { "transform": "@nestia/sdk/lib/transform" }
    ]
  }
}
```

2.3. Broken Assumption

Question	Old TypeScript	TypeScript-Go
source language	TypeScript	TypeScript
compiler implementation	JavaScript	Go
plugin loading	JavaScript module in-process	no old JS host
transformer access	Program, Checker, emit hook	bridge required
result	generated artifacts	typia/nestia engine at risk

2.4. Project Risk

typia

- `createIs<T>()` needs erased type facts
- no transformer, no specialized predicate
- no generated serializer path

nestia

- controllers need compile-time extraction
- no transformer, no SDK contract
- no OpenAPI or mockup pipeline

Product failure

- packages can still install
- source can still look valid
- generated value disappears

2.4. Project Risk

Wait

- upstream plugin model
- unknown timing

Retreat

- runtime schemas
- duplicate source of truth

Shrink

- remove generated features
- lose the reason users came

Build

- new transformer host
- survive on TypeScript-Go

3. Transformer Survival

- 3.1. Compiler Front Door
- 3.2. Plugin Host
- 3.3. Transformer Lifecycle
- 3.4. Runtime and Editor Path

3.1. Compiler Front Door

Missing piece

- no patched JavaScript host
- no plugin entry point

Required host

- load the project
- expose compiler facts
- run transformers

Design choice

- explicit front door
- tsc-shaped command

3.1. Compiler Front Door

Keep TypeScript-Go

- native compiler base
- semantic analysis
- diagnostics

Add missing layer

- transformer host
- plugin execution path
- emit integration

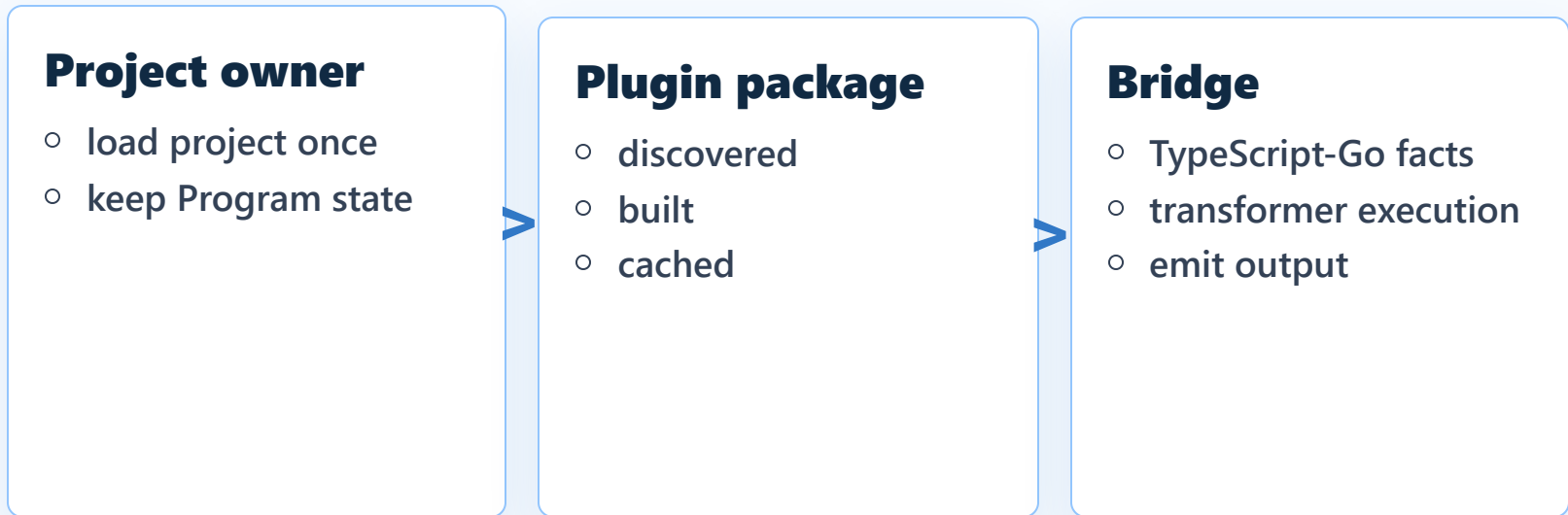
Goal

- keep products alive
- move with the compiler

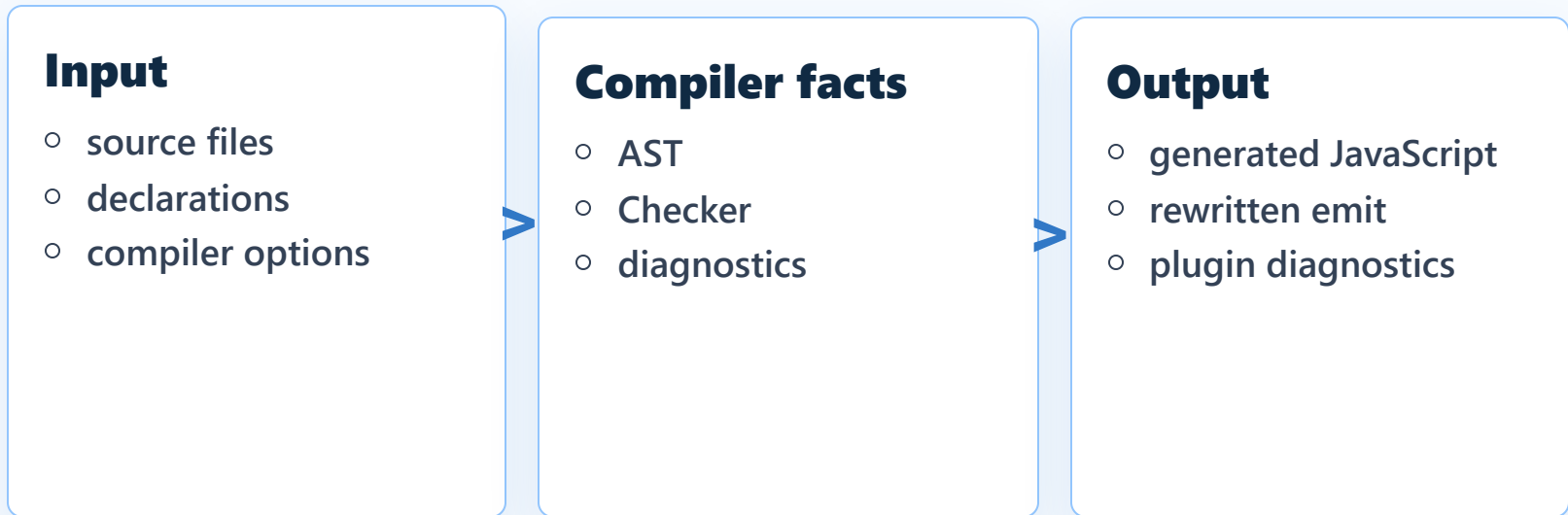
3.2. Plugin Host

Old	TTSC
Patch TypeScript install	Explicit compiler front door
JS compiler process	TypeScript-Go base
JS transformer loaded directly	Plugin host bridge
Build-time only	Build, runtime, editor

3.2. Plugin Host



3.3. Transformer Lifecycle



3.3. Transformer Lifecycle

typia user

- `typia.createIs<T>()`
- no runtime schema

nestia user

- `@TypedRoute`
- generated SDK

Internal shift

- old patch removed
- TTSC host added

3.4. Runtime and Editor Path

```
npx ttsx src/index.ts
```

tsx convenience

- direct TypeScript execution

Type safety

- real check
- plugin-aware path

Avoid

- transpile-only blind spot

3.4. Runtime and Editor Path

@ttsc/vscode

- plugin diagnostics
- code actions
- plugin commands

VS Code Plugin

- project view
- early feedback

Goal

- CI confirms
- editor discovers

3.4. Runtime and Editor Path

Build

- `ttsc`
- transformer emit

Runtime

- `ttsx`
- checked execution

VS Code Plugin

- `@ttsc/vscode`
- plugin diagnostics

One toolchain

- not one wrapper

4. Toolchain Opportunity

- 4.1. Compiler State
- 4.2. @ttsc/lint
- 4.3. @ttsc/graph
- 4.4. From Patch to Toolchain

4.1. Compiler State

Already loaded

- Program
- AST
- Checker
- diagnostics

Expensive to rebuild

- parse
- module graph
- type services

Reusable by tools

- transformers
- linter
- graph

4.1. Compiler State

Load once

- project
- options
- dependencies



Analyze once

- semantic graph
- diagnostics
- type relations



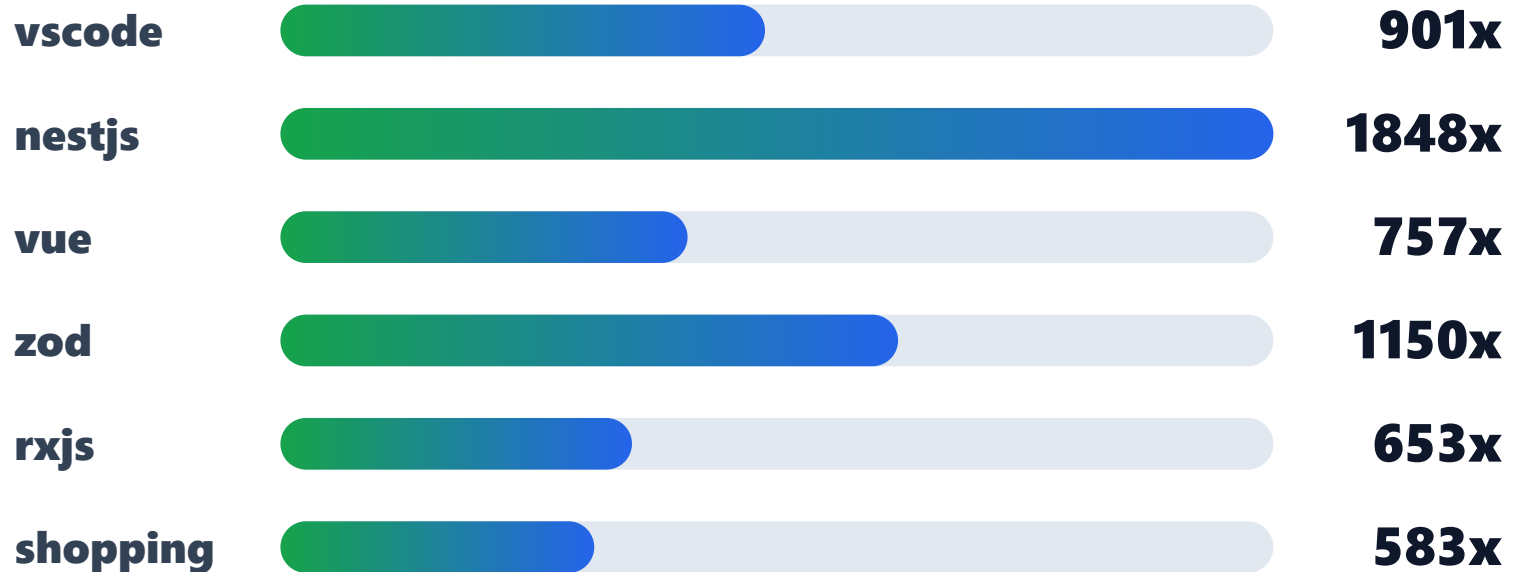
Reuse many times

- emit
- lint
- graph

4.2. @ttsc/lint

Isolated lint speedup

Legacy TypeScript + ESLint baseline versus @ttsc/lint.



4.2. @ttsc/lint

Compiler gate

Lint rules fail the same build path as type errors.

Diagnostic shape

Every finding is rendered as an `error TSxxxxx`.

One stream

Terminal, CI, and editor consume the same output.

src/index.ts

```
var x: number = 3;
let y: number = 4;
const z: string = 5;

console.log(x + y + z);
```

npx ttsc --noEmit

```
$ npx ttsc --noEmit

src/index.ts:3:7 - error TS2322
Type 'number' is not assignable to type 'string'.

src/index.ts:2:5 - error TS17397 [prefer-const]
Use const instead of let.

src/index.ts:1:1 - error TS11966 [no-var]
Unexpected var, use let or const instead.
```

4.3. @ttsc/graph

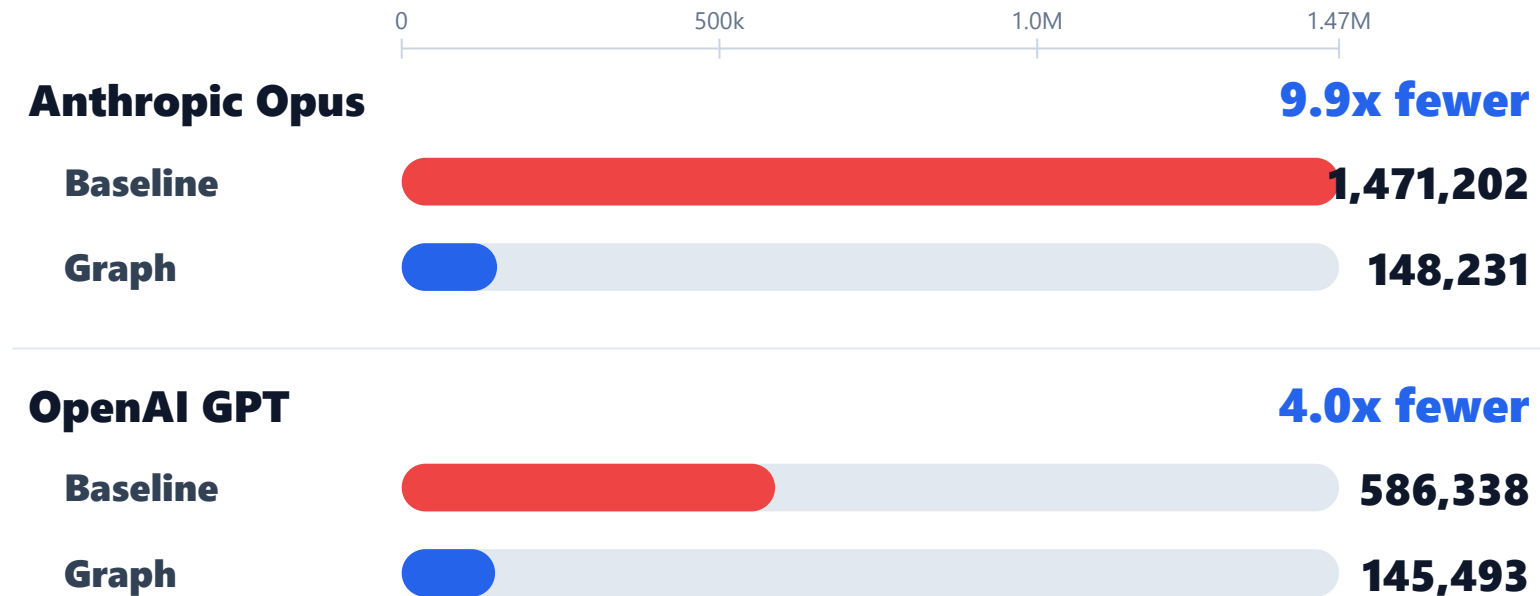
- **Checker facts**
 - declarations
 - resolved edges
 - diagnostics
- **Agent queries**
 - overview
 - query
 - trace / expand
- **Goal**
 - stop guessing from text



4.3. @ttsc/graph

TypeORM agent token usage

Common token axis. Graph stays near 145k, baseline changes by model.



4.4. From Patch to Toolchain

Started as survival

- keep typia alive
- keep nestia alive
- keep transformers alive



Became infrastructure

- compiler front door
- plugin host
- shared compiler state



Opened new tools

- lint
- graph

4.4. From Patch to Toolchain

Compiler

- `ttsc`

Runtime

- `ttsx`

VS Code Plugin

- `@ttsc/vscode`

Reuse

- `@ttsc/lint`
- `@ttsc/graph`

Closing

TypeScript-Go

- faster compiler
- new runtime substrate

Transformers

- old patch point gone
- host required

TTSC

- transformer survival
- compiler-state toolchain

Outcome

- user API stays TypeScript
- backend tooling moves forward

Closing

TypeScript v7

- official TypeScript-Go era

TTSC

- released alongside TypeScript v7

Goal

- transformer compatibility
- shared compiler-state toolchain

Q&A

TypeScript Backend Meetup

2026-06-26

Samchon